Université Sultan Moulay Slimane
Ecole Supérieure de Technologie
- Beni Mellal -

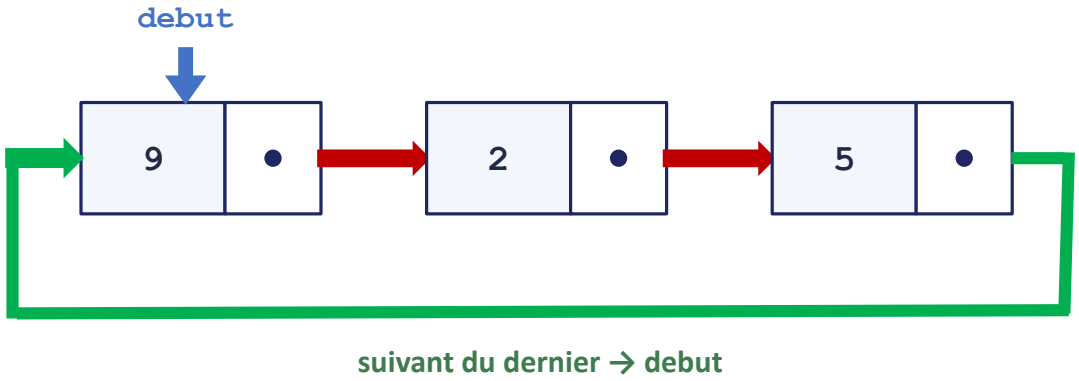
STRUCTURES DE DONNÉES

Chapitre 5 : Listes Circulaires

Filière : DUT GI
Semestre : S3
Année Universitaire : 2026/2027

Pr. M. OUTANOUTE

L'idée : un cercle, pas une ligne



Le ***suivant*** du *dernier nœud* ne vaut plus *NULL* — il pointe vers ***debut***. La liste n'a plus de fin visible.

PARTIE A — LSC Circulaire

Le piège central

Le parcours du Chapitre 2 (LSC) (***while (courant != NULL)***) ne s'arrête jamais sur une liste circulaire — il n'existe plus de *NULL*. Boucle infinie.

```
Noeud *courant = debut;
do {
    // traiter courant
    courant = courant->suivant;
} while (courant != debut);
```

Le coût réel de la circularité

Sans repère sur le dernier nœud, ***insererEnTete*** doit le retrouver par un parcours complet.

Au Chapitre 2, cette opération était en temps constant — ici, elle devient O(n).

C'est le prix à payer pour la circularité, sans second repère.

precedent résout le coût

```
Noeud *dernier = debut->precedent;    // O(1), sans parcours
```

Dans une *liste circulaire doublement chaînée*, ***debut->precedent*** EST toujours *le dernier nœud*.

insererEnTete redevient O(1), comme au Chapitre 2.

Le fil narratif : le tour de passage

L'ordre de passage des exposés oraux :

Chaque étudiant présente à son tour, et une fois tous passés, on reprend au premier — un scénario naturellement circulaire.

Récapitulatif

- Liste circulaire : le ***suivant*** *du dernier nœud* pointe vers ***debut***, pas *NULL*.
- Partie A (LSC), bâtie sur le Chapitre 2. Partie B (LDC), bâtie sur le Chapitre 4.
- Le piège central : condition d'arrêt sur ***debut*** (***do...while***), pas sur *NULL*.
- Le coût de la Partie A : ***insererEnTete*** redevient O(n), faute de repère sur le *dernier nœud*.
- Le gain de la Partie B : ***debut->precedent*** est le *dernier nœud* — O(1) retrouvé.
- Fil narratif : le tour de passage des exposés oraux.

PARTIE A — LSC Circulaire

Déclaration, et le test estVide

```
typedef struct noeud {
    int valeur;
    struct noeud *suivant;
} Noeud;

Noeud *debut = NULL;

int estVide(void) { return debut == NULL; }
```

PARTIE A — LSC Circulaire

Parcours — LE piège

1) Si la *liste est vide*, rien à faire.
2) Sinon, partir de **debut**, traiter *sa* ***valeur***, puis avancer.
3) Répéter tant que le *nœud* ***courant*** n'est pas revenu à ***debut***.

```
void afficherListe(void) {
    if (debut == NULL) { printf("\n"); return; }
    Noeud *courant = debut;
    do {
        printf("%d  ", courant->valeur);
        courant = courant->suivant;
    } while (courant != debut);
}
```

do...while, pas ***while*** : l'action doit se faire avant de vérifier — ***debut*** lui-même doit être traité.

PARTIE A — LSC Circulaire

Insertion en tête — le coût

1) Créer un ***nouveau*** *nœud*, lui donner *sa* ***valeur***.
2) Si la *liste est vide*, le ***nouveau*** *nœud* pointe vers *lui-même*, et devient ***debut***.
3) Sinon, trouver le *dernier nœud* (celui dont ***suivant*** vaut ***debut***).
4) Relier le ***nouveau*** *nœud* entre le *dernier nœud* et l'*ancien* ***debut***.
5) ***debut*** devient le ***nouveau*** *nœud*.

```
void insererEnTete(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    if (debut == NULL) { nouveau->suivant = nouveau; debut = nouveau; return; }
    Noeud *dernier = debut;
    while (dernier->suivant != debut) { dernier = dernier->suivant; }
    nouveau->suivant = debut;
    dernier->suivant = nouveau;
    debut = nouveau;
}
```

PARTIE A — LSC Circulaire

Insertion en fin — presque identique

Étapes identiques, sauf la dernière : ***debut*** n'est pas réaffecté.

```
void insererEnFin(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    if (debut == NULL) { nouveau->suivant = nouveau; debut = nouveau; return; }
    Noeud *dernier = debut;
    while (dernier->suivant != debut) { dernier = dernier->suivant; }
    nouveau->suivant = debut;
    dernier->suivant = nouveau;
}
```

insererEnTete et ***insererEnFin*** partagent le même squelette — seule la ligne ***debut = nouveau;*** les distingue.

PARTIE A — LSC Circulaire

Suppression en tête

1) Si la *liste est vide*, rien à faire.
2) Si *un seul élément* (***debut->suivant == debut***), le libérer, ***debut*** devient *NULL*.
3) Sinon, trouver le *dernier nœud*, avancer ***debut***, puis relier le *dernier* au *nouveau* ***debut***.
4) Libérer l'*ancien nœud*.

```
void supprimerEnTete(void) {
    if (debut == NULL) { return; }
    if (debut->suivant == debut) { free(debut); debut = NULL; return; }
    Noeud *dernier = debut;
    while (dernier->suivant != debut) { dernier = dernier->suivant; }
    Noeud *ancienDebut = debut;
    debut = debut->suivant;
    dernier->suivant = debut;
    free(ancienDebut);
}
```

PARTIE A — LSC Circulaire

Suppression d'un élément donné

```
void supprimerValeur(int valeur) {
    if (debut == NULL) { return; }
    if (debut->suivant == debut) {
        if (debut->valeur == valeur) { free(debut); debut = NULL; }
        return;
    }
    if (debut->valeur == valeur) { supprimerEnTete(); return; }
    Noeud *precedent = debut;
    Noeud *courant = debut->suivant;
    while (courant != debut && courant->valeur != valeur) {
        precedent = courant; courant = courant->suivant;
    }
    if (courant != debut) {
        precedent->suivant = courant->suivant;
        free(courant);
    }
}
```

PARTIE A — LSC Circulaire

Détruire la liste, et le bilan de la Partie A

```
void detruireListe(void) {
    while (debut != NULL) {
        supprimerEnTete();
    }
}
```

Le bilan de la Partie A :

La circularité résout un problème (plus de *fin* à atteindre) mais en crée un autre (***insererEnTete*** en O(n)).
La Partie B résout ce second problème.

PARTIE B — LDC Circulaire

Déclaration

```
typedef struct noeud {
    int valeur;
    struct noeud *suivant;
    struct noeud *precedent;
} Noeud;

Noeud *debut = NULL;
```

Nœud du Chapitre 4 (LDC) — **suivant** et **precedent**.

PARTIE B — LDC Circulaire

Insertion en tête — O(1), le gain

1) Créer un ***nouveau*** *nœud*.
2) Si la liste est vide, le ***nouveau*** *nœud* pointe vers lui-même, et devient ***debut***.
3) Sinon, ***debut->precedent*** EST déjà le *dernier nœud* — aucun parcours.
4) Relier le ***nouveau*** *nœud* entre ce *dernier nœud* et l'*ancien* ***debut***.
5) ***debut*** devient le ***nouveau*** *nœud*.

```
void insererEnTete(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    if (debut == NULL) {
        nouveau->suivant = nouveau; nouveau->precedent = nouveau;
        debut = nouveau; return;
    }
    Noeud *dernier = debut->precedent;
    nouveau->suivant = debut; nouveau->precedent = dernier;
    dernier->suivant = nouveau; debut->precedent = nouveau;
    debut = nouveau;
}
```

PARTIE B — LDC Circulaire

Insertion en fin — aussi O(1)

Étapes identiques, sauf la dernière : ***debut*** n'est pas réaffecté.

```
void insererEnFin(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    if (debut == NULL) {
        nouveau->suivant = nouveau; nouveau->precedent = nouveau;
        debut = nouveau; return;
    }
    Noeud *dernier = debut->precedent;
    nouveau->suivant = debut; nouveau->precedent = dernier;
    dernier->suivant = nouveau; debut->precedent = nouveau;
}
```

Suppression en tête — aussi O(1)

```
void supprimerEnTete(void) {
    if (debut == NULL) { return; }
    if (debut->suivant == debut) { free(debut); debut = NULL; return; }
    Noeud *ancienDebut = debut;
    Noeud *dernier = debut->precedent;
    debut = debut->suivant;
    debut->precedent = dernier;
    dernier->suivant = debut;
    free(ancienDebut);
}
```

PARTIE B — LDC Circulaire

Suppression d'un élément donné

```
void supprimerValeur(int valeur) {
    if (debut == NULL) { return; }
    Noeud *courant = debut;
    do {
        if (courant->valeur == valeur) {
            if (courant->suivant == courant) {
                free(courant); debut = NULL; return;
            }
            courant->precedent->suivant = courant->suivant;
            courant->suivant->precedent = courant->precedent;
            if (courant == debut) { debut = courant->suivant; }
            free(courant); return;
        }
        courant = courant->suivant;
    } while (courant != debut);
}
```

PARTIE B — LDC Circulaire

Détruire la liste, et le bilan du chapitre

```
void detruireListe(void) {
    while (debut != NULL) {
        supprimerEnTete();
    }
}
```

Le bilan du chapitre :

la Partie B combine deux gains déjà vus séparément — la ***circularité*** et le ***double chaînage*** — et leur combinaison élimine le coût que la circularité seule imposait.

Récapitulatif

- Partie A : parcours avec **do...while** (**debut**), **insererEnTete**/**insererEnFin** en O(n).
- Partie A : cas particulier omniprésent — le nœud unique qui pointe vers lui-même.
- Partie B : **debut->precedent** est le *dernier nœud* — insertions en O(1).
- Partie B : ***supprimerValeur*** n'a besoin que d'un seul pointeur, comme au Chapitre 4 (LDC).
- Les deux parties partagent le même squelette ***detruireListe***, réutilisant ***supprimerEnTete***.

L'énoncé

Partie A — version simplement chaînée.

1) Écrire ***inscrireExpose*** (insertion en fin) et ***passerAuSuivant*** (avance ***debut*** vers ***debut->suivant***).
2) Dérouler l'inscription de 3 étudiants (101, 102, 103), puis deux appels à ***passerAuSuivant***.

Partie B — version doublement chaînée.

1) Réécrire ***inscrireExpose*** pour la version doublement chaînée, en O(1).
2) Expliquer, en une phrase, pourquoi cette version est plus rapide.

PARTIE A — LSC Circulaire

Le code

```
void inscrireExpose(int numero) {
    insererEnFin(numero);
}

void passerAuSuivant(void) {
    if (debut == NULL) { return; }
    debut = debut->suivant;
}
```

passerAuSuivant *ne fait aucun* ***free*** *— elle déplace seulement* ***debut****.*

La circularité rend cette opération triviale, même au-delà du dernier étudiant.

PARTIE A — LSC Circulaire

La trace

Étape	Action	État de la liste
1	`inscrireExpose(101)`	**101** (→101)
2	`inscrireExpose(102)`	**101 → 102** (→101)
3	`inscrireExpose(103)`	**101 → 102 → 103** (→101)
4	`passerAuSuivant()`	**102 → 103 → 101** (→102)
5	`passerAuSuivant()`	**103 → 101 → 102** (→103)

debut se déplace sur la liste — elle-même ne change jamais de taille.

PARTIE B — LDC Circulaire

Le code, et la comparaison

```
void inscrireExpose(int numero) {
    insererEnFin(numero);
}
```

Réponse attendue :

En Partie A, ***insererEnFin*** doit parcourir toute la liste pour trouver le dernier nœud.

En Partie B, ***debut->precedent*** donne ce dernier nœud directement, sans aucun parcours

— l'insertion devient O(1) au lieu de O(n).

Exercice

Une équipe de surveillance se relaie en boucle, représentée par une liste circulaire simplement chaînée (globale). Chaque membre de l'équipe est représentée par ses informations :
un ***identifiant***, un ***nom*** et un ***prénom***.

1) Écrire ***ajouterAuTour***, qui insère un membre en fin de tour.
2) Écrire ***personneDeGarde***, qui retourne l'identifiant actuellement en tête, sans rien modifier.
3) Écrire ***finDeService***, qui fait avancer ***debut*** vers la personne suivante, sans rien supprimer.
4) Dérouler l'ajout de 3 membres, puis deux appels à ***finDeService***, et donner le résultat de ***personneDeGarde*** à chaque étape.